

Solving AEOSSP with CP and Local Search

Siyi Song 2400010816

December 16, 2025

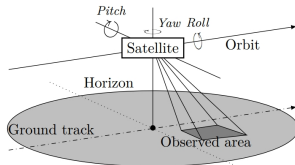
- Introduction and Problem Statement
- Related Work
- A Local Search Approach
- Experiments and Results

Introduction and Problem Statement

Background Information

- AEOSSP

Agile Earth Observation Satellite Scheduling Problem



■ **Figure 1** Schematic view of an agile satellite's observation capabilities.

NP-hard; growing size of instances and constellations;
profitability; Solved ASAP;

Introduction and Problem Statement

Problem and Notations

- Problem

maximize the profit, by scheduling observations and data transfers

- Notations

$\mathcal{R}$: a set of requests; $r \in \mathcal{R}$: a request; $\mathcal{P}_r$: the set of the repetitions.

$\mathcal{S}$: set of satellites; $\mathcal{A}$: set of acquisitions (opportunity)

$\mathcal{A}_{r,t,m}$: the set of possible acquisition of the request r for the period t and the mesh m ;

$\mathcal{A}^s$: the subset of acquisitions which can be made by satellite s

p_a : duration to make the observation (during time window $[r_a, d_a]$);

v_a : profit; γ_a : memory consumption;

$T(a_1, a_2)$: transition time between two acquisitions

$\mathcal{W}_a$: set of possible download windows for each a , where $w = [r_w, d_w]$ during which a can be downloaded.

γ_w : total capacity; C_s : capacity of each satellite

x_a : a boolean selection variable; $s_a \in \{r_a, d_a - p_a\}$: start time variable

$y_{a,w}$: another boolean selection variable

Introduction and Problem Statement

Problem Statement

- Constraints expressed with the notations mentioned above

- Acquisition time windows

$$x_a \implies r_a \leq s_a \leq d_a - p_a \quad \forall a \in \mathcal{A}$$

- Acquisition no-overlap

$$x_a \wedge x_b \implies s_a + p_a + T(a, b) \leq s_b \vee s_b + p_b + T(b, a) \leq s_a \quad \forall \{a, b\} \subseteq \mathcal{A}^s$$

- Acquisitions alternatives

$$\sum_{a \in \mathcal{A}_{r,t,m}} x_a \leq 1 \quad \forall r \in \mathcal{R}, \forall t \in \mathcal{P}_r, \forall m \in \mathcal{M}_r$$

- Acquisitions download

$$\sum_{w \in \mathcal{W}_a} y_{a,w} \leq x_a \quad \forall a \in \mathcal{A}$$

$$y_{a,w} \implies s_a + p_a \leq r_w \quad \forall a \in \mathcal{A}, \forall w \in \mathcal{W}_a$$

- Satellite memory

$$\sum_{a \in \mathcal{A}^s | x_a \wedge s_a < t} \gamma_a - \sum_{a \in \mathcal{A}^s, w \in \mathcal{W}_a | y_{a,w} \wedge d_w < t} \gamma_a \leq C_s \quad \forall t$$

- Download capacity

$$\sum_{a \in \mathcal{A}} y_{a,w} \gamma_a \leq \gamma_w \quad \forall w \in \mathcal{W}$$

- Objective

$$\text{maximize the total profit } \sum_{a \in \mathcal{A}} x_a v_a$$

- Simplification
NP-hard; Too complex taking into account all possible extensions.
- Resolution Methods
 - Exact Methods
effective on small instances; MILP formulations (for easier scheduling problems); $\implies$ solve sub-problems as a local search move
 - Heuristics and Metaheuristics
(sometimes combined with deep reinforcement learning or machine learning) also on small instances; Genetic algorithm
 - Local Search and Large Neighborhood Search (LNS)
random choices and greedy algorithms; ruin-and-recreate

A Local Search Approach

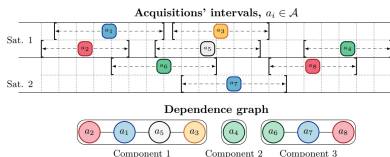
- Two sub-problems

- Acquisition planning: selection and allocation of the acquisitions using local search method

For each $s \in \mathcal{S}$ define a dependence graph whose nodes are $a \in \mathcal{A}^s$.

There is an edge between two acquisitions $\{a, b\} \subseteq \mathcal{A}^s$ if

$d_a + T(a, b) > r_b \vee d_b + T(b, a) > r_a$, i.e. when their time windows overlap while considering transition time.



Possible sequences: $\bar{g}_1 = [a_3 \rightarrow a_5]$, $\bar{g}_2 = \emptyset$, $\bar{g}_3 = [a_6 \rightarrow a_7 \rightarrow a_8]$

■ **Figure 2** Example of sequences defined from the dependence graph built from a set of acquisitions. Acquisitions with the same colour are alternatives to cover the same mesh for the same task of the same request. The interval graph contains three connected components.

- Download planning: one planning of file downloads using greedy algorithm

A Local Search Approach

Acquisition Planning

- Greedy initial solution construction (from scratch)
 - use $U(a)=v_a/p_a$ to measure utility
 - iterate over acquisitions whose associated mesh and task are not currently covered by descending order of utility
 - insert them into the plan to minimize the increment of transition times
 - $\mathcal{A} = a_0, a_1, \dots, a_k, a_{k+1}$: a feasible sequence of acquisitions, a_0, a_{k+1} dummy acquisitions, $r_{a_0} = 0, d_{a_{k+1}} = \infty$; find $i \in [0, k]$, minimize $T(a_i, a) + T(a, a_{i+1})$
 - starting time: $s_{a_i} = \max(r_{a_i}, s_{a_{i-1}} + p_{a_{i-1}} + T(a_{i-1}, a_i)) \quad \forall 1 \leq i \leq k$
 - slack(time space between actual finishing time and deadline):
 $slack(a_i) = d_{a_i} - s_{a_i} - p_{a_i} \quad \forall 1 \leq i \leq k$
 - gap(time space for being too early and have to wait):
 $gap(a_i) = s_{a_i} - s_{a_{i-1}} + p_{a_{i-1}} + T(a_{i-1}, a_i)$
 - cgap(total waiting time): $cgap[a_i] = \sum_{j=0}^{i-1} gap(a_j)$
 - time length available between a_i and a_j :
 $F(a_i, a_j) = cgap[a_j] - cgap[a_i] + slack(a_j)$
 - time length available for task insertion: $I[a_i] = \min_{i < j \leq k} F(a_i, a_j)$
 - minimize transition time satisfying the constraint of $I[a_i]$

A Local Search Approach

Acquisition Planning

• Local Search

- optimize the plan by recomputing sub-sequences
- orienteering problem over the *complete graph*
- The graph: node(acquisition with the same time window, with a profit); path(length: transition time)
- find a path, maximize the sum of the profit of the visited nodes within the time limit.
- Profit increment: swap a for b $P(a) = v_a - \sum_{b \in \mathcal{A}_{r,t,m} \setminus a} x_b v_b$
- in decreasing order of marginal profit, insert, until the first acquisition with a non positive marginal profit.
 - minimize transition times
 - call an external solver by rescheduling the whole connected component

A Local Search Approach

Acquisition Planning

- Local Search

- use *Tempo*(a CP-SAT) to solve TSTPW: traveling salesman problem with time windows (shortest path & considering the time window limit)
- time window, transition.
- only constraint: no overlap on all the acquisitions.
- no objective function, just a valid sequence; limited by a fail limit
- main benefit: reorder the sequence when a solution is found, leaving space for further acquisition insertions.
- choose the first improving among all randomly ordered neighbors until a local minimum is reached.

A Local Search Approach

Download Planning

- computing a download plan by assigning time windows
 - greedy algorithm iterates over download windows by increasing order of starting time until capacity is reached or there is no remaining.
 - sort acquisitions on board at the start of the time window, and sort them by decreasing order of memory consumption.
- fixing potential remaining memory overloads by deselecting acquisitions
 - a memory overload repairing procedure: since memory constraints may be violated
 - knapsack problem: capacity, weights, values.
- download planning: not an issue, often an easy task, rarely a bottleneck
- some images may be downloaded in the next stage, because of the rolling horizon.

Experiments and Results

Experimental setting

- Evaluate against state-of-the-art commercial solver and recent proposals from the literature.
- Instances
 - Squillaci et al.: 32 instances containing up to 1140 requests.
 - a stereoscopic request: bond the second and the first together.
 - transition time: euclidean distance between the targets

■ **Table 1** Satellites orbital parameters for the simulation, including plane inclination i (deg), altitude a (km), longitude of ascending node Ω (deg), mean anomaly M and period P (min).

(a) Inclined plane orbits.

i	a	Ω	M	P
90	500	-180	$\{-180, -90, 0, 90\}$	90
90	500	-90	$\{-180, -90, 0, 90\}$	90
90	500	0	$\{-180, -90, 0, 90\}$	90
90	500	90	$\{-180, -90, 0, 90\}$	90

(b) Heliosynchronous orbits.

i	a	Ω	M	P
98	550	-144	-180	90
98	550	-60	-90	90
98	550	60	0	90
98	550	144	90	90

Experiments and Results

Experimental setting

- Instances

- Produced by an instance generator we designed
 - input: coordinates corresponding to POIs; $\implies$ by greedy algorithm, meshes cover them; simulate orbits, determine time windows; randomly selecting a subset of the meshes
 - for each POI set, 4 instances from 50 to 500 requests.

Experiments and Results

Comparison Points

- LNS: use destructive phases that randomly select a subset of connected components, then relax a subset of acquisitions within to have more choice to reinsert a request after relaxation
- CP Model with CPO: CPO runs LNS until a local optimum is detected $\implies$ complete search

Experiments and Results

Results and Discussion

Comparison with LNS and CPO

■ **Table 2** Profit and gap of the solutions found by CPO, LNS and LS-tempo over Squillaci et al.'s instances. Optimal profit is noted with a star (*). For the sake of clarity, only instances where at least one solver has not found the optimal solution are shown.

R	OS	V	S	P	CPO		LNS		LS-tempo	
					profit	gap	profit	gap	profit	gap
500	500	0	0	0	5070*	0.00	5069	<0.01	5070*	0.00
1000	1000	0	0	0	10165	0.13	11382	0.03	11536	0.01
500	0	500	0	0	21176	0.01	20988	0.02	21200	0.01
1000	0	1000	0	0	40280	0.39	52024	0.21	55367	0.16
250	0	0	0	250	4100	0.03	4018	0.05	3946	0.07
500	0	0	0	500	10856	0.12	11072	0.10	10512	0.15
1000	0	0	0	1000	14643	0.33	15415	0.29	13953	0.36
570	120	150	270	30	22379	0.02	22409	0.02	22585	0.01
1140	240	300	540	60	13075	0.19	16223	<0.01	16220	<0.01
500	500	0	0	0	9270*	0.00	9263	<0.01	9267	<0.01
1000	1000	0	0	0	13409	0.26	17920	0.02	18055	0.01
250	0	250	0	0	18500*	0.00	18456	<0.01	18500*	0.00
500	0	500	0	0	36592	0.01	36304	0.02	36420	0.02
1000	0	1000	0	0	51732	0.29	67868	0.07	68564	0.06
250	0	0	0	250	6445*	0.00	6442	<0.01	6388	0.01
500	0	0	0	500	12024	0.10	12307	0.08	11662	0.13
1000	0	0	0	1000	17410	0.35	19385	0.27	18055	0.32
570	120	150	270	30	22132	<0.01	22152	<0.01	22187	<0.01
1140	240	300	540	60	31666	0.29	43828	0.02	43899	0.02
Average					0.134		0.064		0.070	

■ **Table 3** Profit and gap of the solutions found by CPO, LNS and LS-tempo over our instances. Optimal profit is noted with a star (*).

Set	R	CPO		LNS		LS-tempo	
		profit	gap	profit	gap	profit	gap
antilles	50	20104	<0.01	20050.6	0.01	20090	<0.01
	100	36480	0.02	36192.6	0.03	36328	0.02
	250	71324	0.30	73033.8	0.28	74035	0.27
	500	81745	0.60	89858.4	0.56	88222	0.57
sahara	50	17751	0.01	17725	0.01	17743	0.01
	100	38702	0.03	38393.6	0.04	38570	0.03
	250	73972	0.32	75885.2	0.30	76020	0.30
	500	76581	0.61	87462.4	0.56	85559	0.57
fountains	50	20304	0.01	20265.4	0.01	20287	0.01
	100	37048	0.02	36897.8	0.02	37014	0.02
	250	90934	0.15	95060.6	0.11	97721	0.08
	500	106102	0.52	141014.6	0.37	141331	0.37
military	50	22467*	0.00	22442.2	<0.01	22467*	0.00
	100	38489*	0.00	38447	<0.01	38489*	0.00
	250	102849	<0.01	101945.6	0.01	102868	<0.01
	500	206813	0.03	207729.2	0.03	211080	0.01
world heritage	50	18222*	0.00	18213.8	<0.01	18222*	0.00
	100	36699	<0.01	36655.4	<0.01	36691	<0.01
	250	103648	0.02	103253.8	0.02	103860	0.01
	500	185196	0.09	192808.4	0.05	197782	0.03
Average		0.136		0.120		0.115	

CPO: small instances; improve with longer time

LS-tempo: good, often best on both instances, apart from periodic requests (little successful insertion using CP-SAT), find the best solutions more often

LNS: most balanced results, good for periodic-only instances, best gap in average.

Experiments and Results

Results and Discussion

● Impact of the CP-SAT Solver

■ **Table 4** Average gap, average number of restart (r), average number of calls to Tempo (t), average success rate when calling Tempo (s), time spent in the solver (Ts) and time ratio spent in modelling the sub-problem with the solver (Tm) for various fail limits. For Squillaci et al. instances, we only used the instances where at least one of the solvers did not find the optimal solution

Fail limit	Squillaci et al. instances						Our instances					
	Gap	r	t	s	Ts	Tm	Gap	r	t	s	Ts	Tm
0	0.089	2507	-	-	-	-	0.115	45772	-	-	-	-
50	0.070	337	153300	47.6%	32%	15%	0.115	5302	1341790	7.68%	21%	54%
100	0.071	264	104603	48.0%	36%	13%	0.115	3047	851625	7.81%	26%	48%
500	0.071	251	122940	50.1%	51%	8%	0.115	5418	1046200	8.10%	41%	40%
1000	0.074	233	114420	50.6%	56%	7%	0.115	4649	946012	8.15%	48%	34%
10000	0.083	208	123406	50.9%	65%	5%	0.116	3665	525106	8.20%	66%	19%
100000	0.089	188	113339	51.5%	69%	5%	0.118	3364	414748	8.26%	73%	14%

- $r \downarrow$, $\text{gap} \downarrow$ over the first instance
- $Ts \uparrow$, $Tm \downarrow$ more on solving, not modeling
- the second instance: almost same gap, many calls, low successful rate, not worth the effort
- a small fail limit improves the gap, no use in increasing the fail limit too much

Experiments and Results

Results and Discussion

- Memory Management Impact

■ **Table 5** Result on our set of instance with memory management

Set	$\mathcal{R}$	CPO		Greedy		LS-tempo	
		profit	gap	profit	gap	profit	gap
antilles	50	20102	<0.01	20089	<0.01	20087	0.01
	100	36323	0.02	36308	0.02	36326	0.02
	250	69674	0.31	74115	0.27	73929	0.27
	500	67979	0.67	89069	0.56	88329	0.57
sahara	50	17748	0.01	17736	0.01	17744	0.01
	100	38517	0.03	38568	0.03	38564	0.03
	250	73805	0.32	76101	0.30	76036	0.30
	500	68814	0.65	86572	0.56	85834	0.57
fountains	50	20301	0.01	20282	0.01	20289	0.01
	100	36939	0.02	36991	0.02	37012	0.02
	250	84771	0.21	97456	0.09	97703	0.09
	500	93629	0.58	142525	0.36	141493	0.36
military	50	22467*	0.00	22467*	0.00	22467*	0.00
	100	38489*	0.00	38488	<0.01	38489*	0.00
	250	100592	0.03	102838	<0.01	102865	<0.01
	500	165914	0.23	211001	0.02	211088	0.01
world heritage	50	18222*	0.00	18217	<0.01	18222*	0.00
	100	36692	<0.01	36686	<0.01	36691	<0.01
	250	98776	0.06	103766	0.01	103856	0.01
	500	152770	0.25	197722	0.03	197794	0.03
Average		0.170		0.115		0.116	

profit & gap taking into account the memory constraints and computing a download plan; CPO lose 3.5%, ours doesn't change

Conclusion and Further Work

- solver for AEOSSP for the constellation of 20 AEOSs by acquisition selection and download planning
- The type of instances determines which solver to use
- Further work should focus on adding different moves and variations to our solver, like the insertion of several acquisitions to improve efficiency by decreasing the number of calls.

Thank you!

Thank you!